A aprendizagem de Máquina - II

Atividade para Compensação de Aula (19.09.2026)
Prof. Luis Claudio

Aluno: Ana Elisa Rubinato Cavalheiro
CDN 6º Semestre

ATENÇÃO:

Após a data limite, haverá desconto de uma falta por dia de atraso.

Situação-Problema 01 — Configuração e treinamento de uma MLP

Uma empresa deseja utilizar uma rede neural para classificar clientes entre duas categorias. Após a preparação dos dados, cada cliente é representado por **20 atributos numéricos**. A equipe construiu inicialmente uma MLP com duas camadas ocultas, contendo **64 e 32 neurônios**, ambas utilizando ReLU, e uma camada de saída com **1 neurônio e Sigmoid**.

O treinamento foi configurado com batch_size = 128, epochs = 40, learning_rate = 0,001 e otimizador Adam. Durante os testes, a equipe percebeu que diferentes configurações produzem resultados distintos e decidiu avaliar alterações tanto na arquitetura quanto no processo de treinamento.

Questões

1) Identifique, entre as configurações apresentadas, quais correspondem a hiperparâmetros de arquitetura e quais correspondem a hiperparâmetros de treinamento. Explique também por que pesos e biases não pertencem a nenhuma dessas duas categorias.

Resposta:

Hiperparâmetros de Arquitetura: Definem a estrutura física e a topologia da rede neural. No caso apresentado, são:

- Quantidade de camadas ocultas (2 camadas).
- Quantidade de neurônios por camada oculta (64 na primeira e 32 na segunda).
- Funções de ativação das camadas ocultas (ReLU) e da camada de saída (Sigmoid).
- Quantidade de neurônios na camada de saída (1 neurônio).

Hiperparâmetros de Treinamento: Definem as condições e a dinâmica sob as quais o aprendizado ocorrerá. No caso apresentado, são:

- Tamanho do lote (batch_size = 128).
- Número de épocas (epochs = 40).
- Taxa de aprendizado (learning_rate = 0,001).
- Algoritmo otimizador (Adam).

Por que pesos e biases não são hiperparâmetros: Pesos (w) e biases (b) são parâmetros internos do modelo. Enquanto os hiperparâmetros são escolhas arbitrárias definidas previamente pelo cientista de dados, os parâmetros são aprendidos e ajustados automaticamente pela própria rede ao longo do treinamento por meio dos dados.

2) A equipe pretende testar diferentes quantidades de camadas ocultas, neurônios e funções de ativação. Explique por que essas escolhas precisam ser avaliadas experimentalmente e como elas podem afetar a capacidade de representação da rede.

Resposta:

A escolha da quantidade de camadas ocultas, do número de neurônios e das funções de ativação precisa ser feita de forma experimental porque não existe uma regra matemática universal que determine a arquitetura ideal previamente. Cada configuração representa uma hipótese inicial de modelagem que deve ser testada e validada com base no desempenho observado em dados de validação. Essas decisões afetam diretamente a capacidade de representação da rede, pois a quantidade de camadas (profundidade) e de neurônios por camada (largura) define a complexidade e o nível de abstração dos padrões que o modelo consegue extrair dos dados. Se a rede for muito pequena, ela pode ter capacidade insuficiente para capturar os padrões relevantes e sofrer sob underfitting; por outro lado, uma rede excessivamente ampla e profunda aumenta drasticamente o custo computacional e o risco de memorização dos dados de treino, levando ao overfitting. Paralelamente, as funções de ativação nas camadas ocultas desempenham um papel fundamental ao introduzir não linearidade no sistema. Sem elas, o empilhamento de sucessivas camadas densas equivaleria a uma única transformação linear, impossibilitando a rede de separar classes com fronteiras curvas ou não lineares. Além disso, a experimentação de diferentes ativações, como a ReLU e suas variantes em substituição à Sigmoide ou Tanh, é indispensável para garantir que o modelo aprenda de forma estável, contornando limitações numéricas como o problema de saturação e o gradiente que desaparece.

3) Considere agora alterações no batch_size, no número de épocas e no learning_rate. Explique como cada uma dessas alterações pode modificar o processo de treinamento, incluindo quantidade de atualizações, estabilidade do gradiente, velocidade de aprendizado e possibilidade de underfitting ou overfitting.

Resposta: As alterações no tamanho do lote (batch size), no número de épocas e na taxa de aprendizado (learning rate) afetam diretamente a dinâmica e a eficácia do treinamento da rede. Ao modificar o batch size, altera-se a quantidade de amostras avaliadas antes de cada ajuste nos parâmetros: a escolha de um lote menor divide o conjunto de dados em mais frações, o que eleva a frequência de atualizações dos pesos por época e permite correções constantes de rota, embora o cálculo do gradiente se torne mais instável e ruidoso por utilizar poucas amostras; em contrapartida, um lote maior estima gradientes mais representativos e estáveis, porém realiza um número menor de atualizações ao longo do mesmo volume de dados. Paralelamente, o número de épocas determina quantas passagens completas a rede realiza sobre todo o conjunto de treinamento: um valor excessivamente reduzido não fornece repetições suficientes para que os pesos convirjam para uma boa solução, resultando em underfitting, enquanto um número muito alto de épocas pode levar o modelo a memorizar detalhes e ruídos das amostras de treino, degradando sua capacidade de generalização por meio do overfitting, além de demandar tempo e processamento

desnecessários. Por fim, a taxa de aprendizado atua regulando a escala do passo dado na atualização dos parâmetros na direção oposta ao gradiente: valores muito altos podem acelerar a fase inicial, mas trazem oscilações bruscas e o risco de saltar sobre a região de perda mínima, inviabilizando a convergência, ao passo que taxas excessivamente baixas tornam os deslocamentos minúsculos, tornando o aprendizado extremamente lento e aumentando a chance de estagnação antes do ponto ideal.

4) Explique, de forma integrada, o papel da função de perda, do gradiente, do learning rate e do otimizador durante o treinamento. Deixe claro por que o gradiente não é um hiperparâmetro.

Resposta: Durante o processo de treinamento, esses elementos atuam de forma totalmente integrada a cada iteração: ao final do *forward pass*, a função de perda (*loss function*) compara a probabilidade prevista com a categoria real do cliente, resumindo a magnitude do erro cometido pelo modelo em um único valor escalar. A partir desse erro, o algoritmo de *backpropagation* calcula o gradiente por meio da regra da cadeia, determinando as derivadas parciais da perda em relação a cada peso e viés individual, o que indica matematicamente a sensibilidade de cada parâmetro e a direção em que o erro cresce com maior intensidade. Em seguida, a taxa de aprendizado (*learning rate*) entra como um fator de escala que controla o tamanho do passo a ser dado na direção contrária a essa subida, enquanto o otimizador atua como o algoritmo que define a regra prática de atualização dos parâmetros, utilizando os gradientes para deslocar os pesos em direção ao ponto de menor erro no caso do Adam, combinando a inércia dos passos passados com o ajuste adaptativo do tamanho do passo para cada peso individualmente. Por fim, destaca-se que o gradiente não é um hiperparâmetro porque não consiste em uma configuração escolhida ou fixada previamente pelo cientista de dados, mas sim em um resultado estritamente matemático derivado de forma analítica e instantânea pelo cálculo das derivadas parciais durante a execução do treinamento.

Situação-Problema 02 — Classificação de imagens utilizando CNN

Uma indústria deseja classificar imagens coloridas de produtos para identificar possíveis defeitos. Cada imagem possui dimensão **32 × 32 × 3**. A primeira camada convolucional utiliza **16 filtros 3 × 3**, stride 1, padding same e função ReLU. A rede possui posteriormente outra camada convolucional, uma operação de Max Pooling e, antes das camadas densas, uma operação Flatten.

Durante o desenvolvimento, alguns integrantes da equipe confundiram conceitos como canais, filtros, mapas de características e quantidade de parâmetros, além de questionarem por que utilizar uma CNN em vez de simplesmente transformar toda a imagem em um vetor e utilizar uma MLP.

Questões

5) Explique como um filtro 3 × 3 atua sobre uma imagem RGB. Indique a dimensão completa de cada filtro, como os três canais participam do cálculo e quantos mapas de características são produzidos pelos 16 filtros.

Resposta: Quando a entrada é uma imagem colorida no padrão RGB, ela possui 3 canais de cor correspondentes ao vermelho, verde e azul, o que faz com que cada filtro não seja uma matriz plana, mas sim um volume tridimensional com dimensão completa de 3 × 3 × 3. Cada filtro possui três conjuntos distintos de pesos 3 × 3, um para cada canal, e realiza convoluções separadas multiplicando elemento a elemento os valores de cada região da imagem pelos respectivos pesos. Em seguida, os resultados dessas três operações são somados entre si juntamente com um único termo de viés (bias) do filtro, e o valor obtido passa pela função de ativação ReLU para gerar uma única ativação naquela posição espacial. Dessa forma, todo o volume de entrada percorrido por um filtro é consolidado em uma única saída bidimensional, fazendo com que cada filtro produza exatamente um mapa de características e, portanto, o conjunto de 16 filtros da camada resulte em um volume final de 16 mapas de características.

6) Calcule a quantidade de parâmetros treináveis dessa primeira camada convolucional. Em seguida, explique por que essa quantidade não aumenta quando a altura e a largura da imagem aumentam, desde que sejam mantidos a quantidade e o tamanho dos filtros e a quantidade de canais.

Resposta: A quantidade de parâmetros treináveis da primeira camada convolucional é calculada considerando que cada um dos 16 filtros tem formato 3 × 3 × 3, totalizando 27 pesos por filtro, aos quais se soma 1 bias por filtro, resultando em 28 parâmetros por filtro e perfazendo 448 parâmetros treináveis no total da camada. Esse número não se altera caso a altura e a largura da imagem aumentem porque as redes convolucionais operam sob o princípio do compartilhamento de pesos (weight sharing). Isso significa que os mesmos pesos e bias contidos no filtro 3 × 3 × 3 são reutilizados ao longo de toda a varredura da imagem, independentemente de sua resolução espacial; se a imagem for maior, o filtro apenas se deslocará por mais

posições para gerar mapas de características maiores, sem exigir novos parâmetros para o aprendizado.

7) Explique como stride e padding influenciam as dimensões espaciais dos mapas de características. Em sua resposta, diferencie same e valid.

Resposta:O stride e o padding regulam diretamente o tamanho da área percorrida e as dimensões espaciais finais dos mapas de características. O stride define a quantidade de passos que o filtro avança a cada deslocamento na horizontal e na vertical, de modo que passos maiores que um reduzem a sobreposição entre janelas e encolhem a altura e a largura do mapa resultante. O padding consiste na inclusão de bordas ao redor da entrada, geralmente preenchidas com zeros, servindo para proteger os pixels periféricos e cantos para que participem dos cálculos com o mesmo peso dos pixels centrais, além de permitir o controle da dimensão de saída. A diferença entre as modalidades reside no fato de que o padding do tipo valid não adiciona nenhuma borda, fazendo com que o filtro atue apenas onde cabe perfeitamente na imagem e gerando um mapa com dimensões menores que a entrada, enquanto o padding do tipo same preenche as bordas com a quantidade de zeros necessária para manter o mapa de características com exatamente as mesmas dimensões espaciais da entrada quando o stride é igual a um, preservando a resolução espacial de 32 × 32 da imagem original.

8) A primeira camada produz 16 mapas de características, que são recebidos pela segunda camada convolucional. Explique qual será a profundidade de cada filtro da segunda camada, quantos mapas de características serão produzidos pelos seus 32 filtros e qual é a função do Flatten antes das camadas densas.

Resposta: Como a primeira camada convolucional gerou 16 mapas de características, esses mapas passam a funcionar diretamente como os 16 canais de entrada da camada seguinte, exigindo que cada filtro da segunda camada tenha profundidade 16 para conseguir cobrir todas as entradas simultaneamente e realizar a combinação de seus valores com pesos específicos para cada canal. Por sua vez, como cada filtro convolucional agrega todas as profundidades de entrada e gera uma única matriz de saída, os 32 filtros presentes nessa segunda camada produzirão exatamente 32 mapas de características. Antes da etapa final de decisão, a operação Flatten atua reorganizando os múltiplos mapas tridimensionais de características em um único vetor linear contínuo, sem realizar transformações matemáticas que descartem informações e sem aprender pesos adicionais, servindo exclusivamente como uma ponte estrutural que adapta o formato espacial dos dados para a entrada das camadas densas que realizarão a classificação.